

HyperLocal Echo SRS

HyperLocal Echo — CLI Content Orchestration Engine

Version 1.0 — Draft for Implementation

July 19, 2026

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Intended Audience
 - 1.4 Definitions
- 2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Functions (Summary)
 - 2.3 User Characteristics
 - 2.4 Constraints
- 3. System Architecture
 - 3.1 High-Level Flow
 - 3.2 Directory Structure (produced by each run)
 - 3.3 Project File Layout **[IMPLEMENTER CHOICE for exact names, structure below is required]**
- 4. Functional Requirements — Core Pipeline
 - 4.1 CLI Interface
 - 4.2 Input Validation
 - 4.3 Prompt Assembly
 - 4.4 Response Parsing
 - 4.5 Fabrication Check (Correctness Requirement)
 - 4.6 File Writing
- 5. Functional Requirements — Expansion Modules
- 6. Non-Functional Requirements
 - 6.1 Technology Stack
 - 6.2 Performance
 - 6.3 Reliability
 - 6.4 Security & Privacy
 - 6.5 Usability
- 7. Dashboard Requirements
- 8. Acceptance Criteria
- 9. Future Considerations (v2, Not In Scope Now)
- Appendix A — Finalized System Prompt (for `promptBuilder.ts`)
- Appendix B — Sample Input for Testing

1. Introduction

1.1 Purpose

This document specifies functional and non-functional requirements for **HyperLocal Echo**, a command-line tool that ingests one raw text asset from a small local business (a voice-note transcript, product description, or flyer draft) and compiles it into a multi-channel marketing content matrix. This SRS is written to be implemented directly by an autonomous coding agent (Codex CLI or equivalent) with no additional clarification required. Every section that leaves a design decision to the implementer is marked **[IMPLEMENTER CHOICE]**; everything else is a hard requirement.

1.2 Scope

HyperLocal Echo is a **local-first CLI application**. In v1.0 it:

- Accepts one raw input asset (text file or piped stdin) plus a business type and optional location.
- Calls the Anthropic Claude API to generate a structured content matrix.
- Writes the matrix to disk as separate, cleanly parseable files.
- Renders a local HTML dashboard summarizing the run (reusing the visual language already approved: dark terminal console aesthetic, signal-orange accent, mint “live” status).
- Supports toggling eight optional “expansion modules” on top of the three core outputs.

Out of scope for v1.0: direct posting to social platforms, email-service-provider integration, multi-user accounts, and billing. These are noted as v2 candidates in §9.

1.3 Intended Audience

A coding agent building this from scratch, and the solo developer (business owner or freelance operator) maintaining it afterward.

1.4 Definitions

Term	Meaning
Core block	One of the three always-generated outputs: SEO Blog Post, Script Studio, Smart Newsletter
Module	One of eight optional additional outputs (\$5)
Run	One invocation of the CLI producing one complete output folder
Asset	The raw input text describing the business’s offer or update
Fabrication	Any generated fact (price, date, address, quote, statistic) not present in the input asset

2. Overall Description

2.1 Product Perspective

HyperLocal Echo is a standalone Node.js (or Python — **[IMPLEMENTER CHOICE]**, see §6.1) CLI tool. It is not a web service; it runs on the user's machine, reads local files, calls the Claude API over HTTPS, and writes local output. No database is required for v1.0 — each run is self-contained in its own timestamped output folder.

2.2 Product Functions (Summary)

1. Accept raw input + business type + optional location via CLI flags or an interactive prompt.
2. Validate the input is non-trivial; reject empty/unusable input with a clear error, no API call made.
3. Send one structured prompt to the Claude API requesting all active blocks/modules in a single response, using strict delimiters.
4. Parse the response into a known block structure.
5. Write each block to its own file inside a per-run output folder.
6. Render an HTML dashboard (static file, opens in any browser) summarizing the run, matching the approved visual design.
7. Support a config file for default business profile (name, type, location, tone) so the user doesn't retype it every run.
8. Support toggling modules on/off per run via flags, defaulting to a configurable set.

2.3 User Characteristics

Non-technical to semi-technical small business owners, or a technical assistant running the tool on their behalf. Comfortable running a CLI command but not writing code. No assumption of API/JSON literacy — all raw model output is fully abstracted away from what the user sees; they only see finished files and the dashboard.

2.4 Constraints

- Must run fully offline except for the single outbound call to the Claude API.
- Must never write a file outside the designated output directory.
- Must never fabricate factual details (§4.5) — this is a correctness requirement, not a style preference, because generated JSON-LD and flyer content may be published as-is.
- API key is read from an environment variable (`ANTHROPIC_API_KEY`) or a local `.env` file — never hardcoded, never logged, never included in output files.

3. System Architecture

3.1 High-Level Flow

```
CLI invocation
|
▼
Input Resolution (flags > interactive prompt > config defaults)
|
▼
Input Validation (non-empty, min length, encoding check)
|
▼
Prompt Assembly (system prompt + business profile + asset + active module list)
|
▼
Claude API Call (single request, block-delimited response format)
|
▼
Response Parser (split on delimiters, validate each block present)
|
▼
File Writer (one file per block, into /output/<run-id>/)
|
▼
Dashboard Renderer (fills the approved HTML template with this run's content)
|
▼
Console Summary (prints file paths + open-dashboard hint)
```

3.2 Directory Structure (produced by each run)

```
output/
2026-07-19_1042_sourdough-workshop/
  blog-post.md
  script-studio.md
  newsletter.md
  modules/
    flyer.md
    sms-blast.txt
    review-replies.md
    hashtags.txt
    headline-ab.md
    posting-calendar.md
    language-pack-<lang>.md
    feedback-notes.md
  dashboard.html
  run-manifest.json
```

`run-manifest.json` records: timestamp, business type, location, input asset (verbatim), active modules, model used, and a `fabrication_check: "passed"` flag (see §4.5) for audit purposes.

3.3 Project File Layout [IMPLEMENTER CHOICE for exact names, structure below is required]

```
hyperlocal-echo/
src/
  cli.ts           # entrypoint, arg parsing
  config.ts        # loads/writes ~/.hyperlocal-echo/config.json
  validate.ts      # input validation rules
  promptBuilder.ts # assembles the system + user prompt
```

```
apiClient.ts          # wraps Claude API call
responseParser.ts     # splits delimited response into blocks
fileWriter.ts         # writes output folder + manifest
dashboard/
  template.html       # the approved dashboard shell
  render.ts          # injects run data into template
tests/
package.json
README.md
```

4. Functional Requirements — Core Pipeline

4.1 CLI Interface

```
hyperlocal-echo run \
  --type "Artisan Bakery" \
  --input "./notes/sourdough.txt" (or --stdin, or omit for interactive prompt) \
  --location "Fort Kochi" (optional) \
  --modules flyer,sms,hashtags (optional, comma list; defaults from config) \
  --lang es (optional, only used if language-pack module active) \
  --out ./output (optional, defaults to ./output)
```

Also required:

```
hyperlocal-echo init      # interactive first-run setup, writes config file
hyperlocal-echo config    # view/edit saved business profile + default modules
hyperlocal-echo open <run-id> # opens that run's dashboard.html in default browser
```

FR-1.1 The CLI must run with only `--type` and `--input` supplied; all else falls back to config or sensible defaults. **FR-1.2** If `--input` is omitted and no stdin is piped, the tool opens an interactive multi-line prompt (terminated by a blank line or EOF). **FR-1.3** Unknown flags produce a clear error and a usage hint; the tool never silently ignores an unrecognized flag.

4.2 Input Validation

FR-2.1 Reject input under 8 words or under 20 characters with: `ERROR: insufficient input – need at least one concrete detail (what, when, or price) to generate content.` No API call is made on this path. **FR-2.2** Strip any HTML/script tags from raw input before it is ever included in a prompt (basic injection hygiene). **FR-2.3** Cap raw input at 4,000 words; longer input is truncated with a warning printed to console, not silently cut.

4.3 Prompt Assembly

FR-3.1 The system prompt sent to the Claude API must be exactly the ruleset specified in Appendix A of this document (the finalized HyperLocal Echo system prompt), with `{{BUSINESS_TYPE}}`, `{{RAW_INPUT}}`, `{{LOCATION}}`, and `{{ACTIVE_MODULES}}` substituted. **FR-3.2** Only modules explicitly active for this run are included in the request — inactive modules must not appear in the prompt at all (saves tokens, avoids unused output). **FR-3.3** The model call must request the response in the exact delimited format defined in Appendix A `$“OUTPUT CONTRACT,”` using `===BLOCK: <NAME>===` headers and `---BLOCK---` separators.

4.4 Response Parsing

FR-4.1 Parser splits the raw model response on `---BLOCK---` and matches each segment to its `===BLOCK: <NAME>===` header. **FR-4.2** If any of the three core blocks (SEO_BLOG, SCRIPT_STUDIO, SMART_NEWSLETTER) is missing from the response, the run fails loudly: `ERROR: model response incomplete – missing <NAME>. No files written.` Partial output is never written to disk. **FR-4.3** If an active module’s block is missing but all core blocks are present, the run succeeds for core blocks and prints a warning listing the missing module(s); it does not fail the whole run. **FR-4.4** The parser must tolerate minor whitespace variance around delimiters (leading/trailing newlines) without failing.

4.5 Fabrication Check (Correctness Requirement)

FR-5.1 After parsing, run a lightweight check: extract any number-like tokens (prices, times, dates, capacities) from each generated block and confirm each also appears, in some form, in the raw input asset. **FR-5.2** Any generated numeric/factual token that cannot be traced to the input is flagged. The run still completes, but: - The console prints: `⚠ WARNING: possible fabricated detail – “<token>” in <block name>. Review before publishing.` - `run-manifest.json` sets `fabrication_check: “flagged”` and lists the flagged strings. **FR-5.3** This check is a heuristic safety net, not a guarantee — the system prompt’s anti-fabrication rule (Appendix A) remains the primary control.

4.6 File Writing

FR-6.1 Each core block and each active module is written to its own file, using the exact filenames in §3.2. **FR-6.2** The output folder name is `<YYYY-MM-DD HHmm> <slug-of-first-five-words-of-input>`. **FR-6.3** The tool never overwrites an existing run folder; a colliding name gets a `-2`, `-3` suffix. **FR-6.4** All files are UTF-8, LF line endings, no BOM.

5. Functional Requirements — Expansion Modules

Each module is independently toggleable. Default-on/off state is configurable in `init`; suggested defaults below.

#	Module	Output file	Default	Requirement
1	Print Flyer + QR	<code>modules/flyer.md</code> + <code>modules/flyer-qr.png</code>	ON	FR-M1: Markdown one-pager sized for print (headline, 3 bullet facts max, CTA, placeholder <code>[QR CODE]</code>). Tool must generate an actual scannable QR PNG locally (no external API) encoding a URL supplied via <code>--cta-link</code> , or a <code>mailto:</code> / <code>tel:</code> fallback if no link given.
2	SMS / WhatsApp Blast	<code>modules/sms-blast.txt</code>	ON	FR-M2: Plain text, hard-capped at 320 characters, no markdown, no links unless <code>--cta-link</code> provided.
3	Review Reply Kit	<code>modules/review-replies.md</code>	ON	FR-M3: Exactly three drafted replies labeled <code>Positive Review Reply</code> , <code>Neutral Review Reply</code> , <code>Negative Review Reply</code> , each under 80 words, referencing the specific event/offer from the input.
4	Hashtag + Geo-Tag Set	<code>modules/hashtags.txt</code>	ON	FR-M4: 15 tags total, grouped under three headers (<code># Neighborhood</code> , <code># Category</code> , <code># Branded</code>), 5 each. No duplicate tags.
5	Headline A/B Pair	<code>modules/headline-ab.md</code>	OFF	FR-M5: Two alternate newsletter subject lines with a one-line rationale each for what's being tested (urgency vs. curiosity, etc.) — rationale only, no performance claims.
6	7-Day Posting Calendar	<code>modules/posting-calendar.md</code>	OFF	FR-M6: Markdown table, 7 rows (Day 1-7 relative to run date), columns <code>Day / Channel / Content Block / Note</code> . Must reference only blocks/modules that were actually generated this run.
7	Language Pack	<code>modules/language-pack-<lang>.md</code>	OFF	FR-M7: Re-renders the three core blocks in the language specified by <code>--lang</code> (ISO 639-1 code). Same facts, no new claims. Errors clearly if <code>--lang</code> omitted while module is active.
8	Feedback Loop	<code>modules/feedback-notes.md</code>	OFF	FR-M8: Only runs if <code>--feedback</code> flag supplies prior metrics (e.g. <code>--feedback "open_rate=18%,click_rate=2%"</code>). Output is a short adjustment note for next run's tone/CTA strength — never fabricates metrics if none supplied; module is skipped with a console note in that case.

6. Non-Functional Requirements

6.1 Technology Stack

[IMPLEMENTER CHOICE] — Recommended: **Node.js 20+ with TypeScript**, using `commander` or `yargs` for CLI parsing, `@anthropic-ai/sdk` for the API call, and a plain template-string or lightweight templating (no heavy frontend framework) for the dashboard HTML. Python + `click` + `anthropic` SDK is an acceptable equivalent if the implementer's environment favors it. Whichever is chosen, keep it a single dependency tree with no build step required for the end user beyond `npm install` / `pip install`.

6.2 Performance

NFR-1 A run with only core blocks completes in under 15 seconds under normal network conditions (excluding the model's own generation latency). **NFR-2** Dashboard rendering (template fill + file write) completes in under 500ms — it is pure string templating, not a build step.

6.3 Reliability

NFR-3 Any API failure (timeout, rate limit, auth error) surfaces a specific, human-readable error — never a raw stack trace to a non-technical user. Raw errors are available via a `--debug` flag. **NFR-4** No partial output folders are ever left behind on failure — write to a temp folder and move-on-success, or delete-on-failure.

6.4 Security & Privacy

NFR-5 API key never appears in `run-manifest.json`, console output, or any written file. **NFR-6** Raw input asset is sent only to the Claude API; no third-party analytics or telemetry calls of any kind in v1.0. **NFR-7** `.env` and `~/.hyperlocal-echo/config.json` must be excluded via `.gitignore` in the generated project scaffold.

6.5 Usability

NFR-8 `hyperlocal-echo run` with no flags at all still works end-to-end via the interactive prompt path (FR-1.2) — a first-time user should never need to read a flags reference before their first successful run. **NFR-9** Every error message states what happened and what to do next in the same sentence.

7. Dashboard Requirements

FR-7.1 `dashboard.html` is a single self-contained static file (inline CSS/JS, no external font/script dependency required to render — Google Fonts may be linked but must degrade gracefully offline to the system-font fallback already defined in the approved template). **FR-7.2** Visual language is fixed to the already-approved design system — implementer must not redesign it:

- Palette: ink `#12151B` background, panel `#1A1F28` / `#20262F`, signal-orange `#FF5A3C` accent, mint `#3DDC97` for “ready/live” states, amber `#F5B942` for tags, hairline borders `#2C323D`.
- Type: display headings in Space Grotesk, body in IBM Plex Sans, code/data/timestamps in IBM Plex Mono.
- Structural motifs: terminal-style top bar with traffic-light dots, an “equalizer” signal-strength signature element, card grid for core blocks, toggle-switch grid for modules, horizontal pipeline strip.

FR-7.3 The dashboard must populate dynamically from that run’s actual generated content (real block previews, real active/inactive module toggle states, real word counts) — not the static mockup copy used during design review. **FR-7.4** `hyperlocal-echo open <run-id>` opens this file in the OS default browser (`open` on macOS, `start` on Windows, `xdg-open` on Linux).

8. Acceptance Criteria

A build is considered complete and correct when all of the following hold:

1. `hyperlocal-echo init` produces a valid config file from an interactive prompt.
2. `hyperlocal-echo run --type "Artisan Bakery" --input "<sample from Appendix B>"` produces all 3 core files plus whichever modules are default-ON, with zero fabricated numeric facts (verified against Appendix B by manual review).
3. Deliberately empty input is rejected per FR-2.1 with no API call made (verifiable via network log / mock).
4. Deliberately malformed model response (simulate by mocking the API) triggers FR-4.2's fail-loud behavior with no partial files written.
5. `dashboard.html` opens correctly in Chrome, Firefox, and Safari with no console errors, and visually matches the approved design system (§7.2).
6. API key set via environment variable never appears in any output file — verified by grepping the entire output folder after a run.
7. All 8 modules produce correct output when individually toggled on in isolation, and correct output when all toggled on simultaneously.
8. Running twice with identical input in the same minute does not overwrite the first run's folder (FR-6.3).

9. Future Considerations (v2, Not In Scope Now)

- Direct publishing integrations (Meta, Google Business Profile, Mailchimp/Klaviyo).
- Multi-business account support with saved profiles.
- A hosted/web version of the dashboard with run history.
- Real performance-metric ingestion (replacing the manual `--feedback` flag with an API pull).
- Batch mode: process a folder of multiple raw assets in one invocation.

Appendix A — Finalized System Prompt (for `promptBuilder.ts`)

You are the core generation engine for "HyperLocal Echo," a CLI tool that converts one raw input asset into a multi-channel marketing matrix for a small local business.

INPUT

- Business Type: `{{BUSINESS_TYPE}}`
- Raw Input Asset: `{{RAW_INPUT}}`
- Optional Location: `{{LOCATION}}` (if absent, keep geography generic)
- Active Modules: `{{ACTIVE_MODULES}}` (comma list, may be empty)

GROUND RULES

1. Use **ONLY** facts present in the Raw Input Asset. Do not invent prices, addresses, hours, awards, review counts, or quotes. If a detail is required for formatting but absent, omit that field rather than fabricate it.
2. Infer tone from the input but always stay professional – no clichés like "hidden gem" or "game-changer."
3. Output **ONLY** the requested blocks below, in order, separated by the exact delimiter line `---BLOCK---`. No preamble, no summary, no closing remarks.

===BLOCK: SEO BLOG===

Markdown blog post, 200-300 words, localized to `{{LOCATION}}` if provided. End with one valid `<script type="application/ld+json">` block, using the most appropriate schema type given the input. Populate only fields supported by the input.

---BLOCK---

===BLOCK: SCRIPT_STUDIO===

Markdown table, 5-8 rows, total runtime 20-45 seconds, columns:
| Timestamp | Visual B-Roll Description | Voiceover Audio |
Directives must be shootable by a non-professional with a phone.

---BLOCK---

===BLOCK: SMART_NEWSLETTER===

Subject Line: `<=55` characters
Preview Text: `<=90` characters
Body: 120-180 words, using `[Customer Name]` placeholder
CTA: one bracketed action label, e.g. `[Reserve Your Spot ->]`

[For each module listed in `{{ACTIVE_MODULES}}`, append the matching block below using the same `===BLOCK: NAME=== / ---BLOCK---` contract.]

===BLOCK: FLYER===

One-pager: headline, up to 3 bullet facts, CTA, `[QR CODE]` placeholder.

===BLOCK: SMS_BLAST===

Plain text, hard cap 320 characters, no markdown.

===BLOCK: REVIEW_REPLIES===

Three replies labeled Positive / Neutral / Negative, each under 80 words.

===BLOCK: HASHTAGS===

15 tags under headers `# Neighborhood / # Category / # Branded` (5 each).

===BLOCK: HEADLINE_AB===

Two subject line variants, each with a one-line rationale.

===BLOCK: POSTING_CALENDAR===

Markdown table, 7 rows: Day / Channel / Content Block / Note.

===BLOCK: LANGUAGE_PACK===

The three core blocks re-rendered in `{{LANG}}`, same facts only.

===BLOCK: FEEDBACK_NOTES===

Short adjustment note based on the supplied prior metrics only.

CONSTRAINTS

- Raw markdown/text only, safely splittable by the delimiter.
- No em dashes; use commas or periods.
- If Raw Input Asset is empty or unusable, output only:
ERROR: insufficient input.

Appendix B — Sample Input for Testing

```
Business Type: Artisan Bakery
Location: Fort Kochi
Raw Input Asset:
"Hey so this Saturday we're doing a sourdough workshop, 10am to noon,
twelve spots, twenty five bucks, includes your own starter jar to
take home."
```

Expected: all numeric facts in output (10am, noon, twelve, \$25) must trace back to this input verbatim or in equivalent phrasing. Any additional number appearing anywhere in the output is a fabrication-check failure.